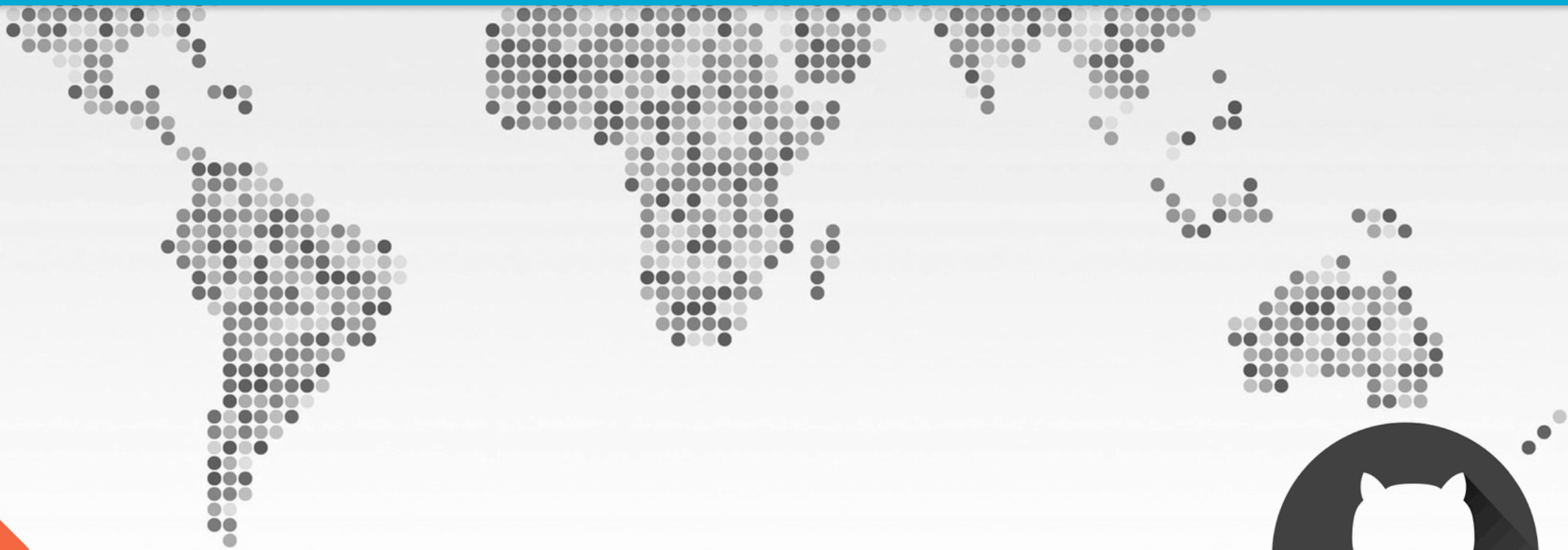


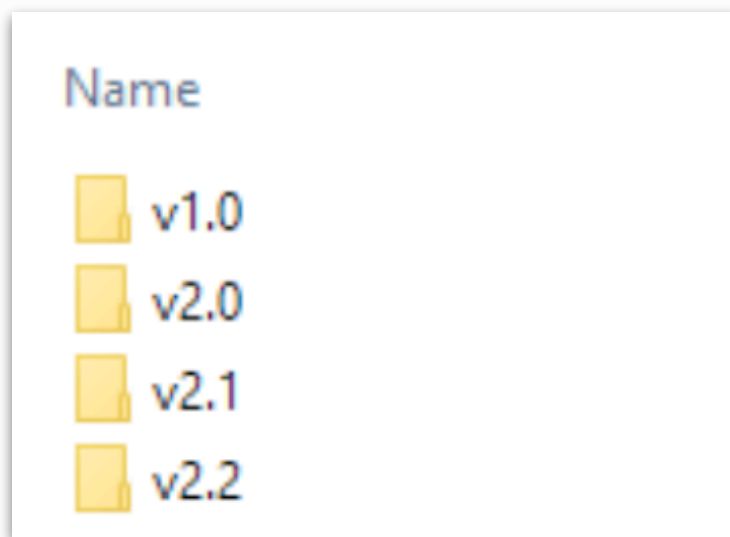
Git e GitHub

Laboratorio di Ingegneria del Software



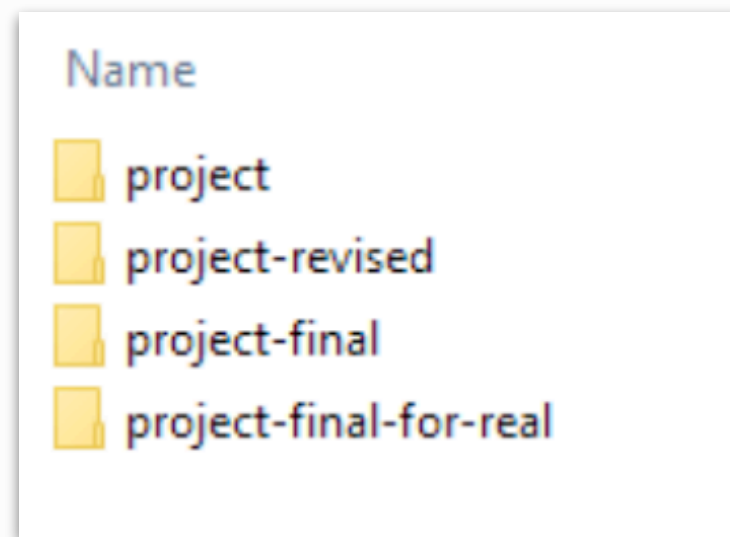
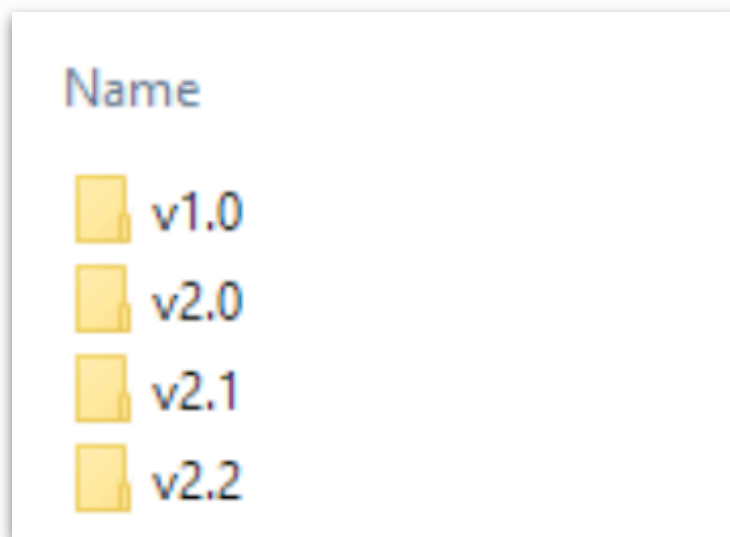
Problema

Vi è mai capitata questa situazione?



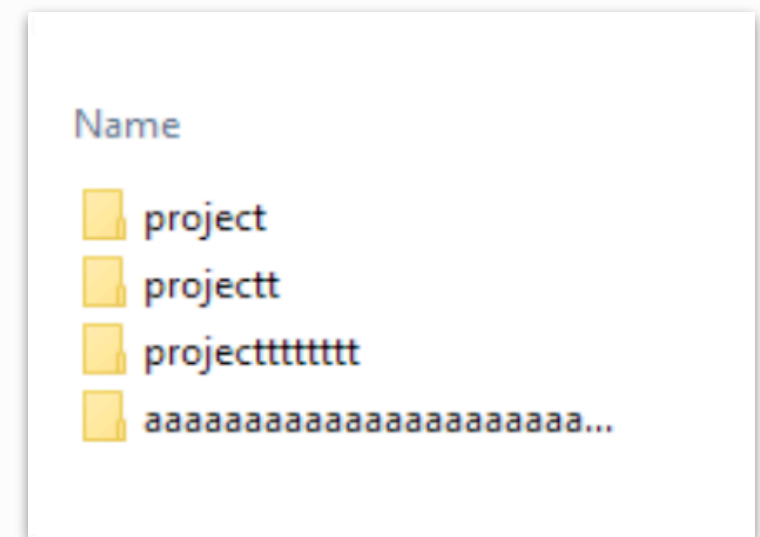
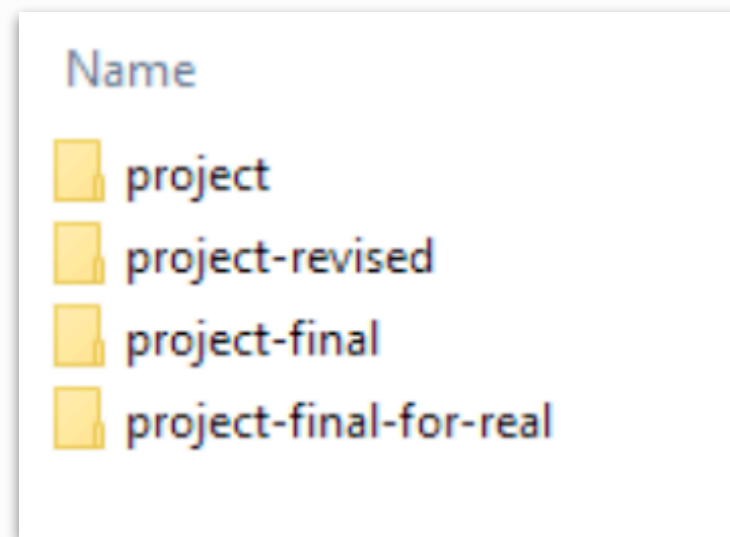
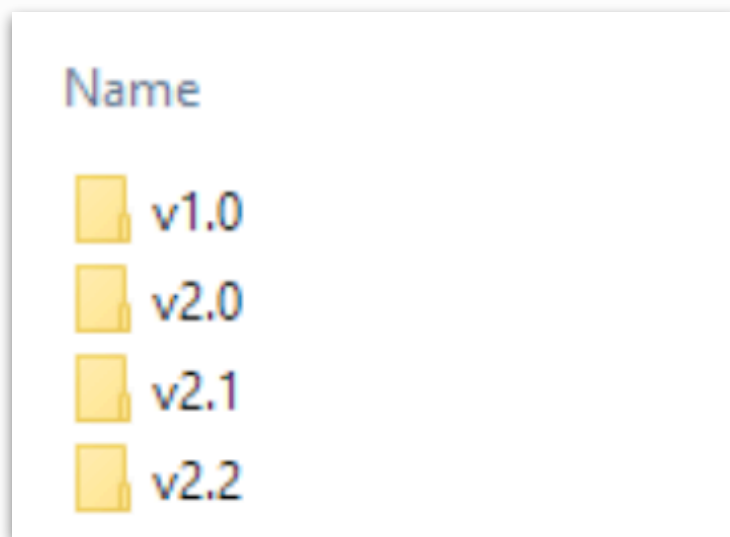
Problema

Vi è mai capitata questa situazione?



Problema

Vi è mai capitata questa situazione?



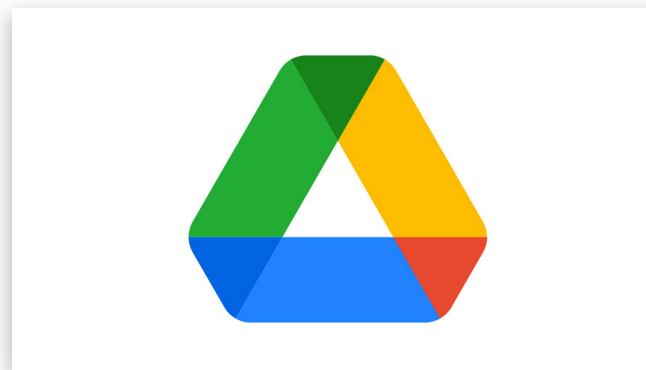
Perché abbiamo bisogno di un modo più semplice per scambiare il codice sorgente?

Se avessimo la necessità di lavorare in team?



Perché abbiamo bisogno di un modo più semplice per scambiare il codice sorgente?

Se avessimo la necessità di lavorare in team?



Perché abbiamo bisogno di un modo più semplice per scambiare il codice sorgente?

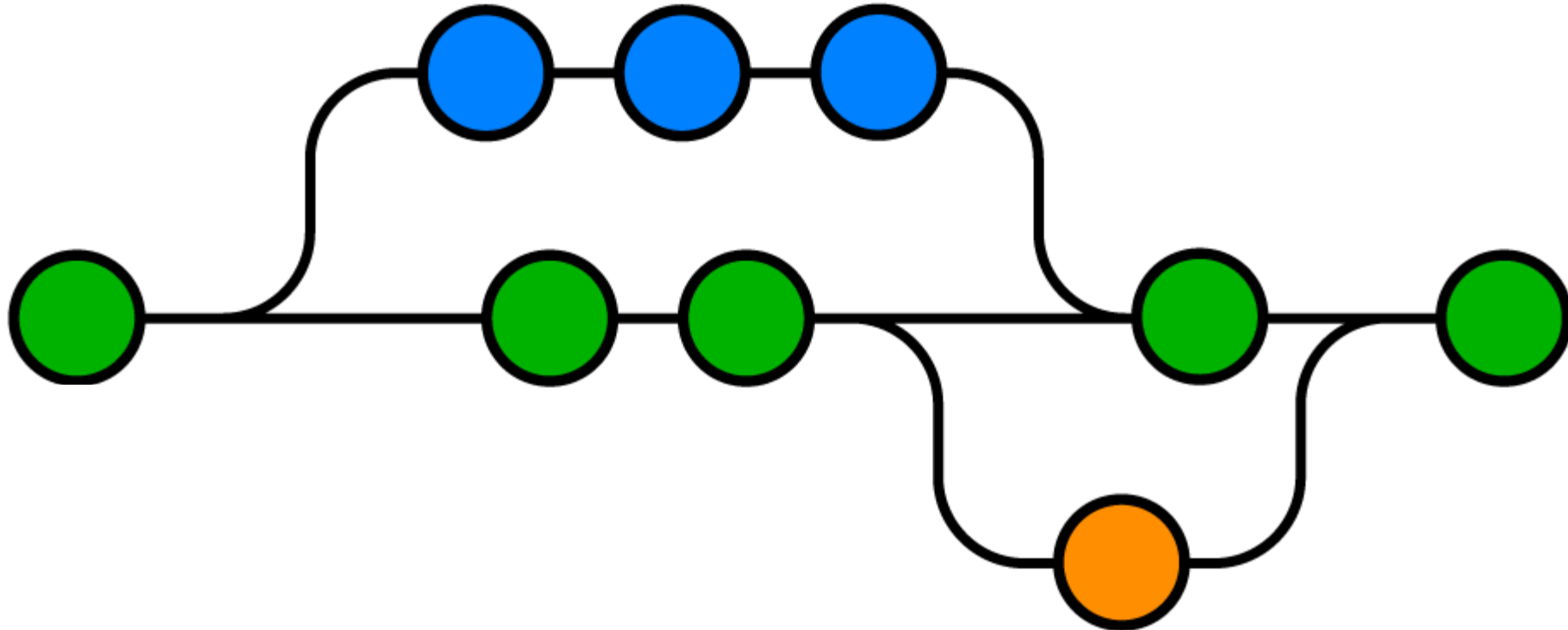
Se avessimo la necessità di lavorare in team?



Tutto il discorso sul version control cadrebbe immediatamente

State-based View: Ogni versione del Configuration Item è identificata dal suo “stato”.

Change-based View: Ogni versione del Configuration Item è identificata dai suoi “cambiamenti”. Andiamo quindi a tracciare l’insieme di cambiamenti che il configuration item ottiene dall’inizio.



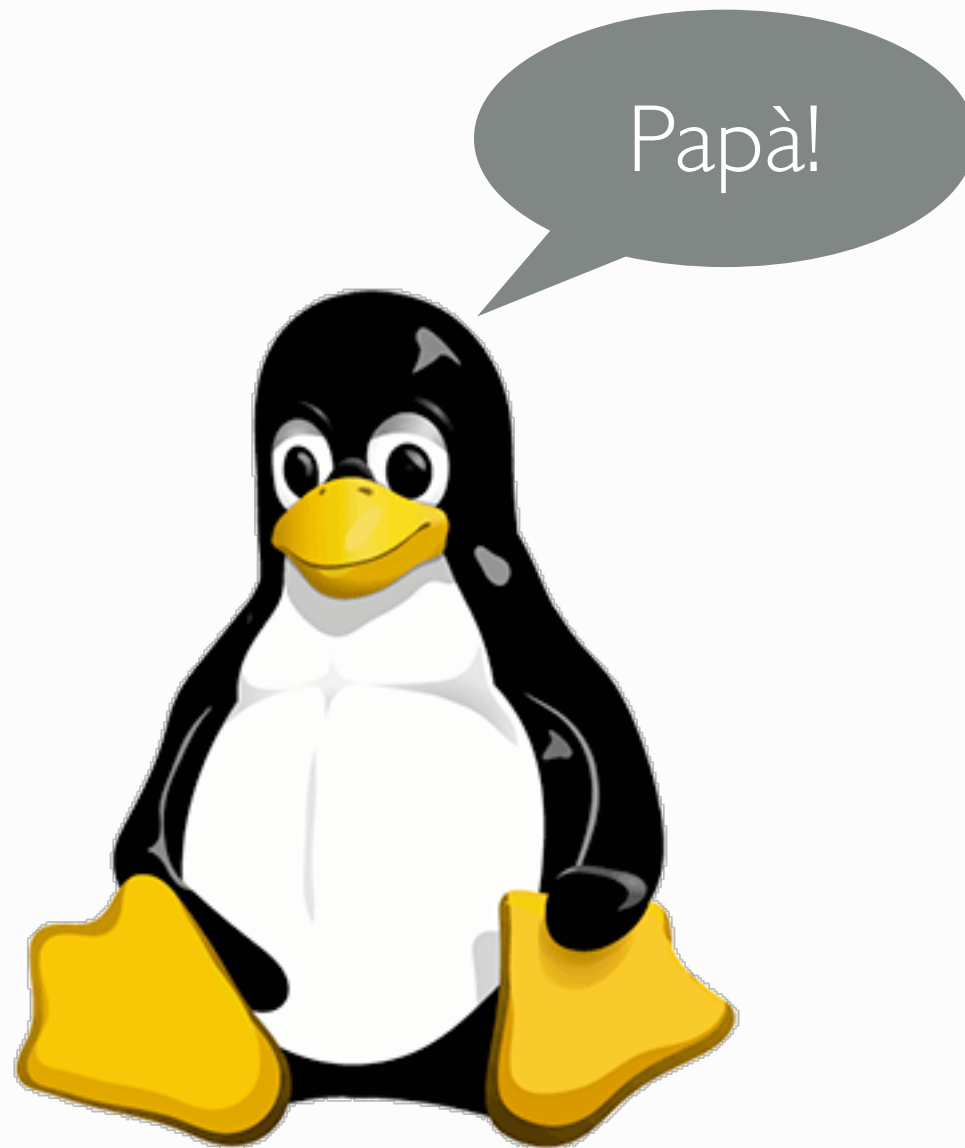
Perché abbiamo bisogno di un modo più semplice per scambiare il codice sorgente?

Se avessimo la necessità di lavorare in team?



Perché abbiamo bisogno di un modo più semplice per scambiare il codice sorgente?

Se avessimo la necessità di lavorare in team?



Perché abbiamo bisogno di un modo più semplice per scambiare il codice sorgente?

Nel lontano 2005 anche Linus Torvalds si trovò davanti allo stesso problema, già all'epoca esistevano meccanismi per il version control, ma nessuno di loro soddisfaceva le sue esigenze....



Perché abbiamo bisogno di un modo più semplice per scambiare il codice sorgente?

Nel lontano 2005 anche Linus Torvalds si trovò davanti allo stesso problema, già all'epoca esistevano meccanismi per il version control, ma nessuno di loro soddisfaceva le sue esigenze....

Domanda

Cosa fa quindi un informatico quando nessun software rispecchia le sue esigenze?



Perché abbiamo bisogno di un modo più semplice per scambiare il codice sorgente?

Nel lontano 2005 anche Linus Torvalds si trovò davanti allo stesso problema, già all'epoca esistevano meccanismi per il version control, ma nessuno di loro soddisfaceva le sue esigenze....

Domanda

Cosa fa quindi un informatico quando nessun software rispecchia le sue esigenze?

Risposta

... Fece il proprio sistema di versioning



Perché abbiamo bisogno di un modo più semplice per scambiare il codice sorgente?

Nel lontano 2005 anche Linus Torvalds si trovò davanti allo stesso problema, già all'epoca esistevano meccanismi per il version control, ma nessuno di loro soddisfaceva le sue esigenze....

Domanda

Cosa fa quindi un informatico quando nessun software rispecchia le sue esigenze?

Risposta

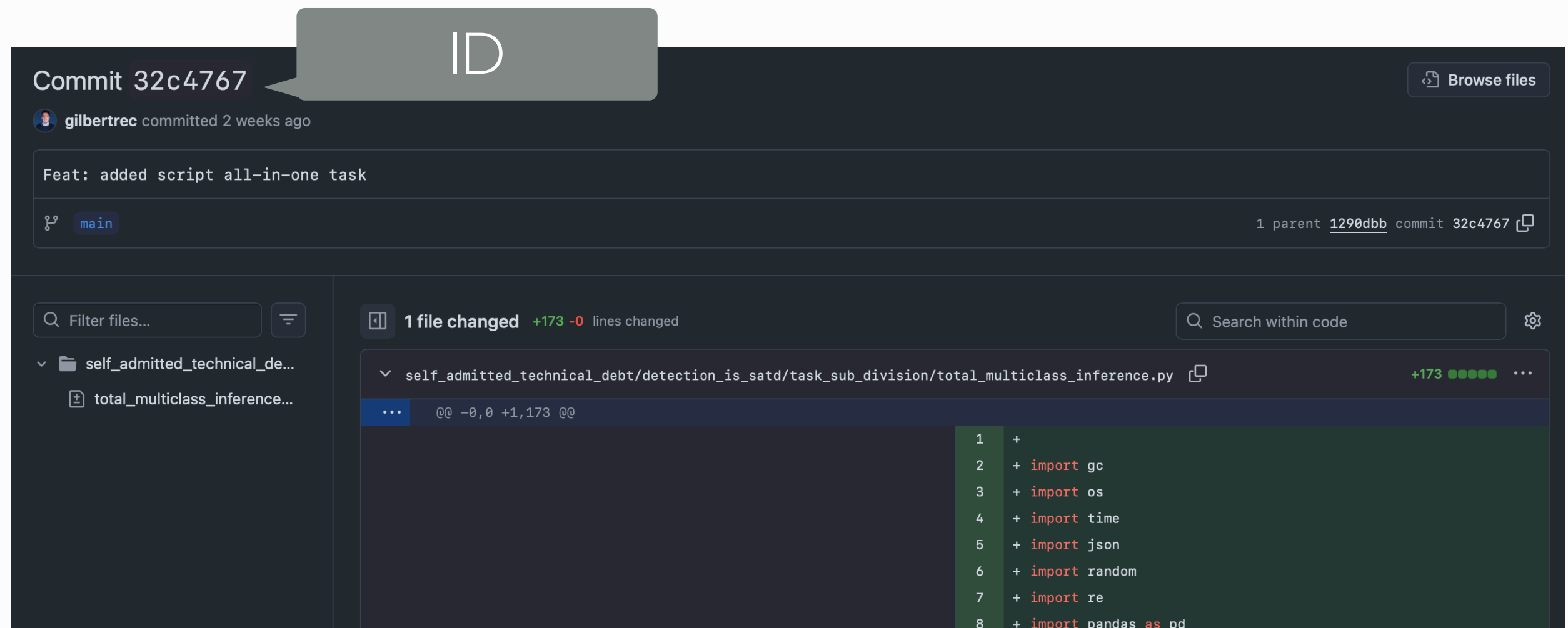
... Fece il proprio sistema di versioning



Cos'è Git?

Git permette sia lo sviluppo in team sia la gestione del sistema di versioning.

Utilizzando Git, ogni volta che si lavora su un progetto e si effettuano dei cambiamenti (e.s. aggiunta/rimozione di file) tali modifiche saranno associate ad un ID e una descrizione. Questo set di operazioni prende il nome di “**COMMIT**”



Cos'è Git?

Git permette sia lo sviluppo in team sia la gestione del sistema di versioning.

Utilizzando Git, ogni volta che si lavora su un progetto e si effettuano dei cambiamenti (e.s. aggiunta/rimozione di file) tali modifiche saranno associate ad un ID e una descrizione. Questo set di operazioni prende il nome di “**COMMIT**”

Commit 32c4767

gilbertrec committed 2 weeks ago

Feat: added script all-in-one task

main

1 parent 1290dbb commit 32c4767

1 file changed +173 -0 lines changed

self_admitted_technical_debt/detection_is_satd/task_sub_division/total_multiclass_inference.py

```
@@ -0,0 +1,173 @@
1 +
2 + import gc
3 + import os
4 + import time
5 + import json
6 + import random
7 + import re
8 + import pandas as pd
```

Cos'è Git?

Git permette sia lo sviluppo in team sia la gestione del sistema di versioning.

Utilizzando Git, ogni volta che si lavora su un progetto e si effettuano dei cambiamenti (e.s. aggiunta/rimozione di file) tali modifiche saranno associate ad un ID e una descrizione. Questo set di operazioni prende il nome di “**COMMIT**”

E' inoltre possibile creare delle versioni diverse dello stesso progetto (es. con funzionalità ancora in sviluppo) (Workspace).

Tali versioni prendono il nome di “**BRANCH**” che metaforicamente si “**staccano**” dal ramo principale chiamato “**MAIN**” o “**Master**” (Master Directories).

Cos'è Git?

Git permette sia lo sviluppo in team sia la gestione del sistema di versioning.

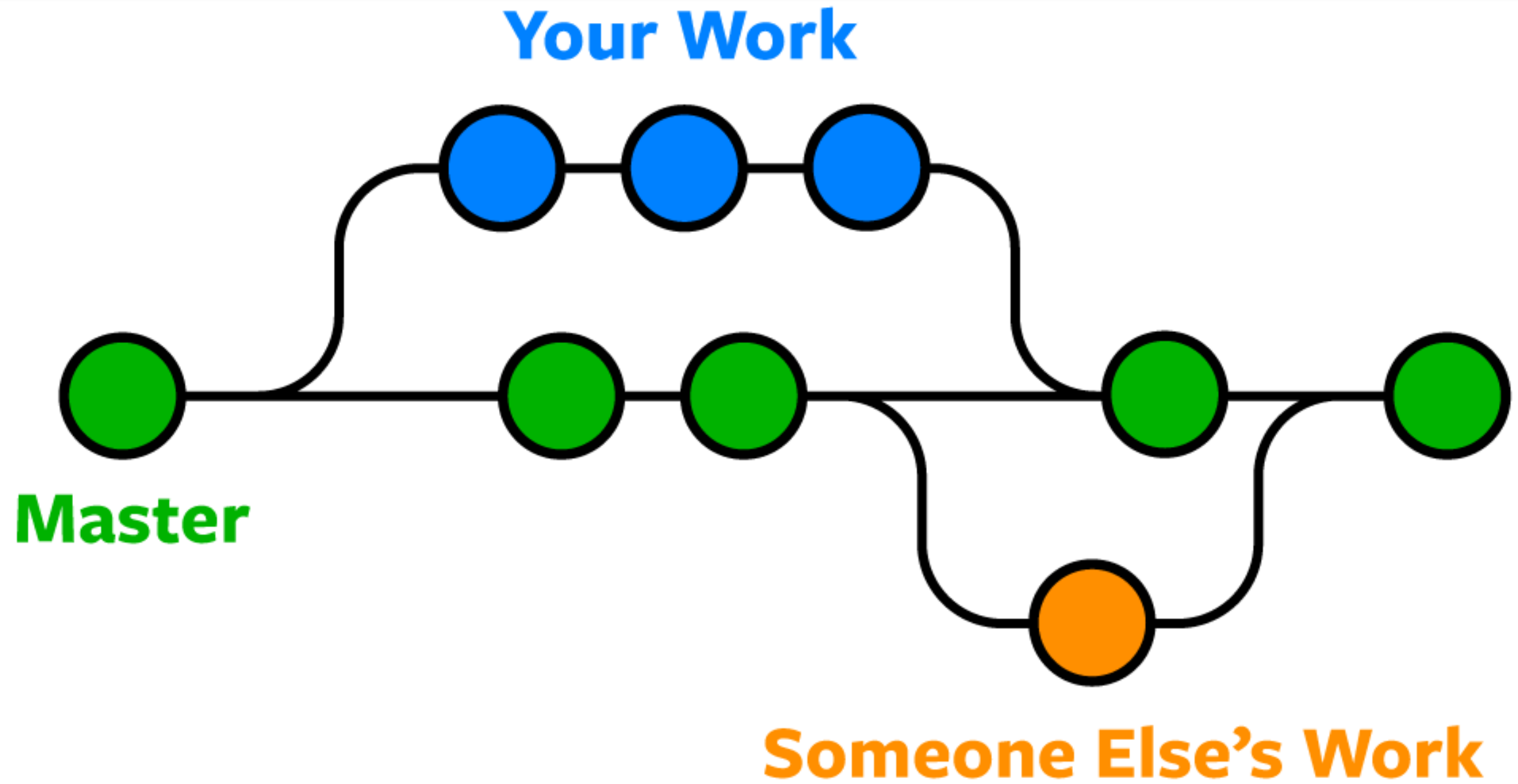
Utilizzando Git, ogni volta che si lavora su un progetto e si effettuano dei cambiamenti (e.s. aggiunta/rimozione di file) tali modifiche saranno associate ad un ID e una descrizione. Questo set di operazioni prende il nome di “**COMMIT**”

E' inoltre possibile creare delle versioni diverse dello stesso progetto (es. con funzionalità ancora in sviluppo) (Workspace).

Tali versioni prendono il nome di “**BRANCH**” che metaforicamente si “**staccano**” dal ramo principale chiamato “**MAIN**” o “**Master**” (Master Directories).

Una volta che si è certi che tali modifiche non hanno compromesso nessuna funzionalità già esistente, allora tale modifica può essere integrata nel branch principale

Workflow di Git



Come installare Git?

Debian/Ubuntu

```
sudo apt update  
sudo apt install git
```

Fedora

```
sudo dnf install git-all
```

MacOS

```
brew install git
```

Assicuratevi di aver installato brew sulla vostra macchina!

Windows

<https://gitforwindows.org/>

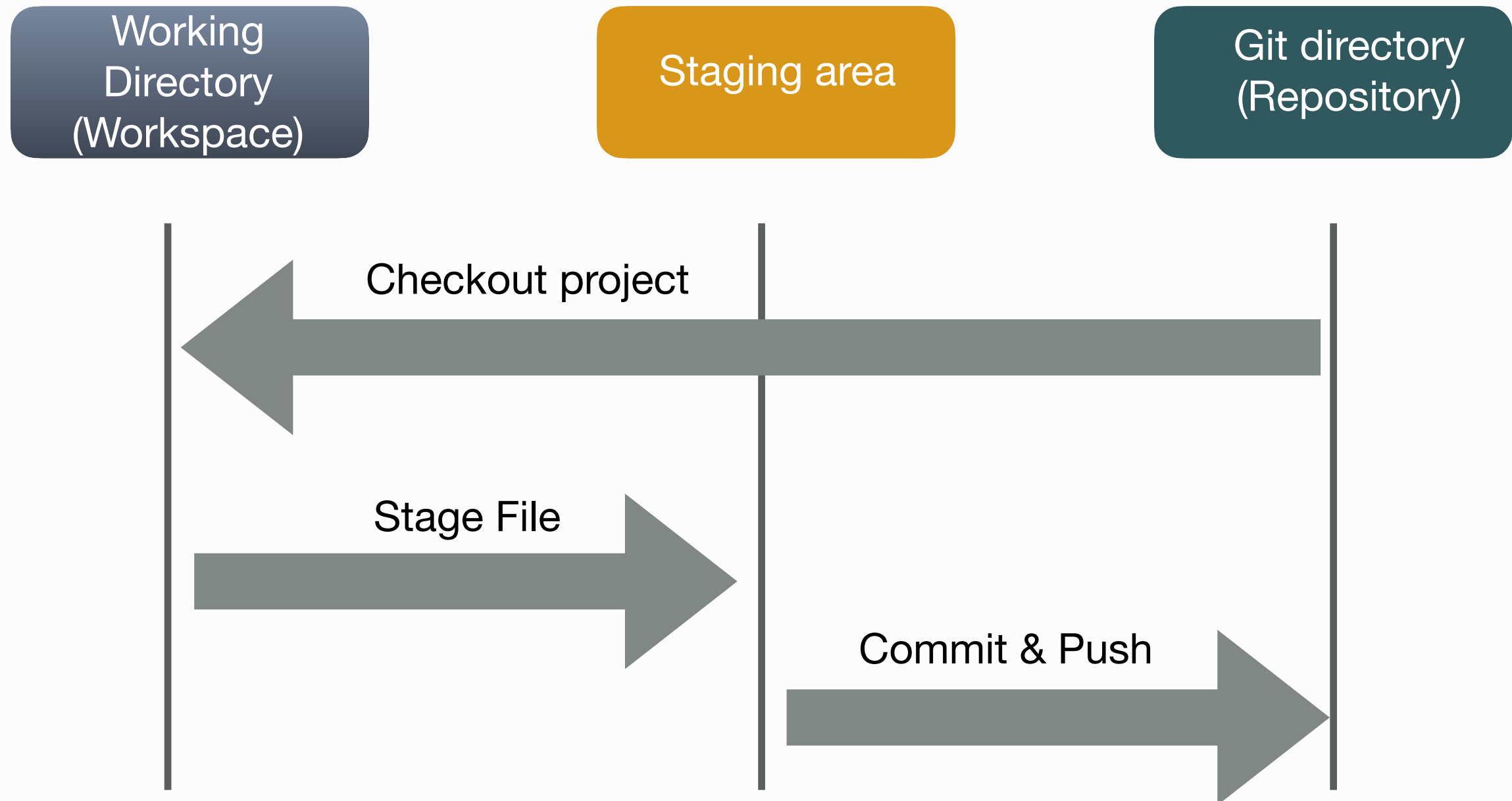
Come installare Git?

Dopo aver scaricato Git, tipicamente verrà chiesto di configurare username e password

```
git config —global user.name "your Name"  
git config —global user.email "example@unisa.it"
```

Queste credenziali si ottengono al momento della registrazione su GitHub, che è una delle piattaforma web che utilizzano Git.

Normal workflow di Git



Caricare file in una repository remota

Dopo aver terminato le modifiche (aggiunta/rimozione/modifica di qualche file), occorre eseguire i seguenti comandi:

```
git add .  
git commit -m "messaggio di commit"  
git push
```

N.B dopo aver eseguito il comando “git add . “ i file modificati saranno in automatico aggiunti nell’area di stage

Solo dopo aver eseguito i comandi di commit e push le modifiche saranno effettivamente caricate sulla repository remota

Come scrivere un buon messaggio di commit?



	feat New feature	feat: add user profile screen
	fix Bug fix	fix: resolve crash on login
	chore Routine tasks / version bump / config	chore: bump version to 1.2.0
	style Code style changes (formatting)	docs: update README with usage examples
	refactor Code restructure (no behavior change)	refactor: move auth logic to new module
	perf Performance improvements	perf: cache API response to improve speed
	test Add or update tests	test: add unit tests for repo module



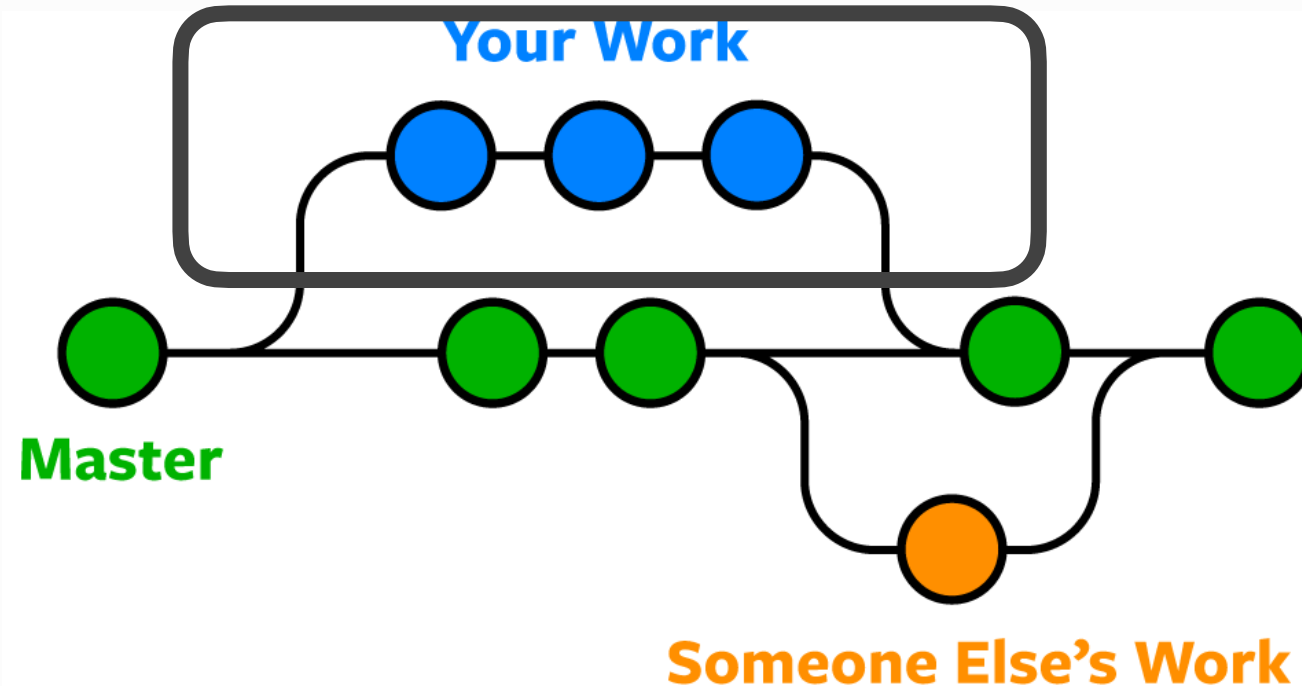
- fixed bug on landing page
- Changed style
- oops
- I think I fixed it this time?
- empty commit messages

E se volessi creare un nuovo branch?

Per creare un nuovo branch ti basterà scrivere a terminale i seguenti comandi:

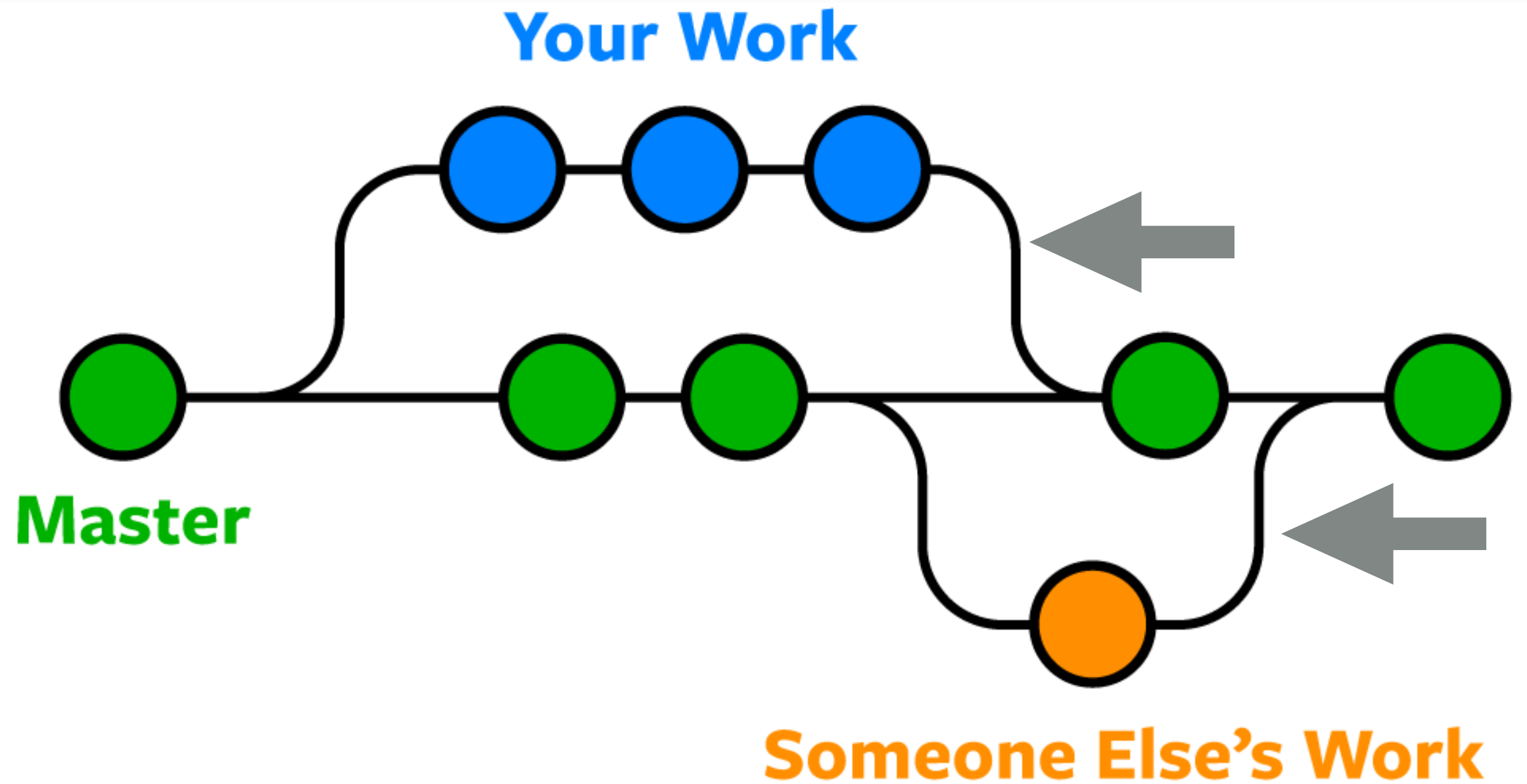
```
git branch <nome del nuovo branch>  
git checkout <nome del branch creato>
```

Con il primo comando creiamo un nuovo branch



Con il secondo comando ci posizioniamo effettivamente sul branch appena creato

Workflow di Git: Cosa succede qui?



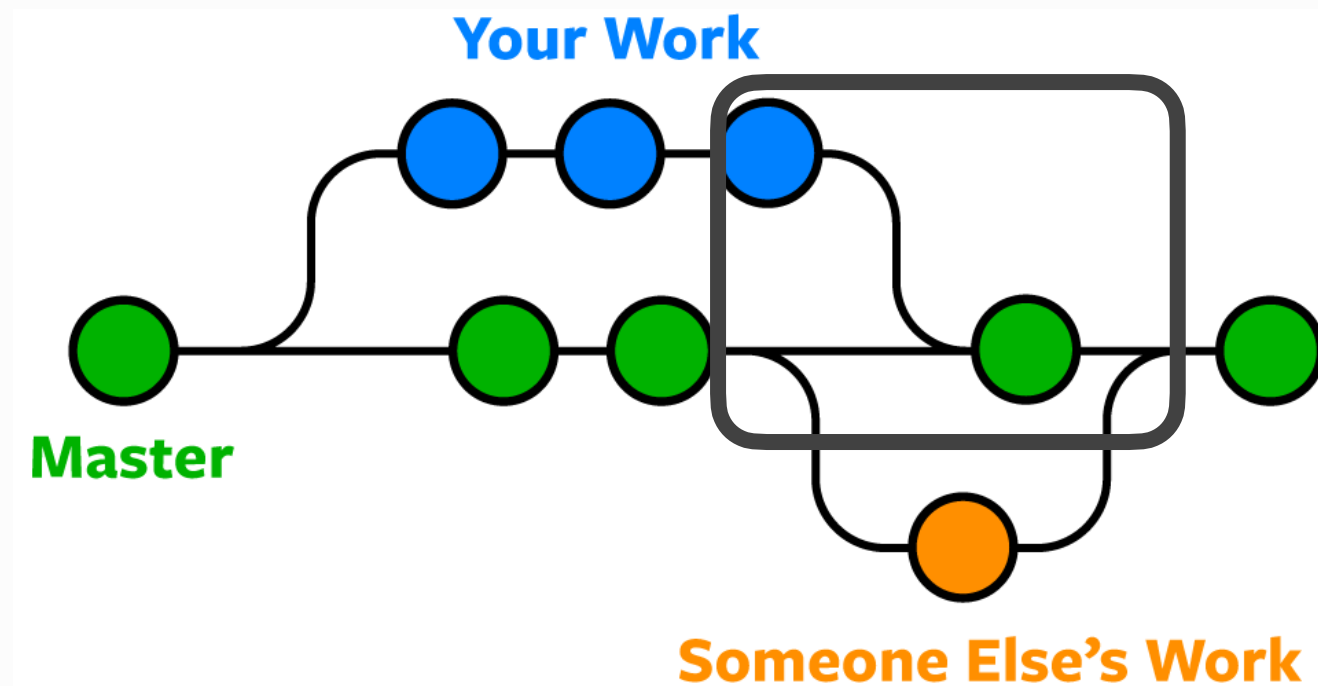
E se volessimo “unire” due branch?



E se volessimo “unire” due branch?

Per “unire” due branch quello che bisogna fare è eseguire il seguente comando:

```
git merge <nome del branch da “unire”>
```



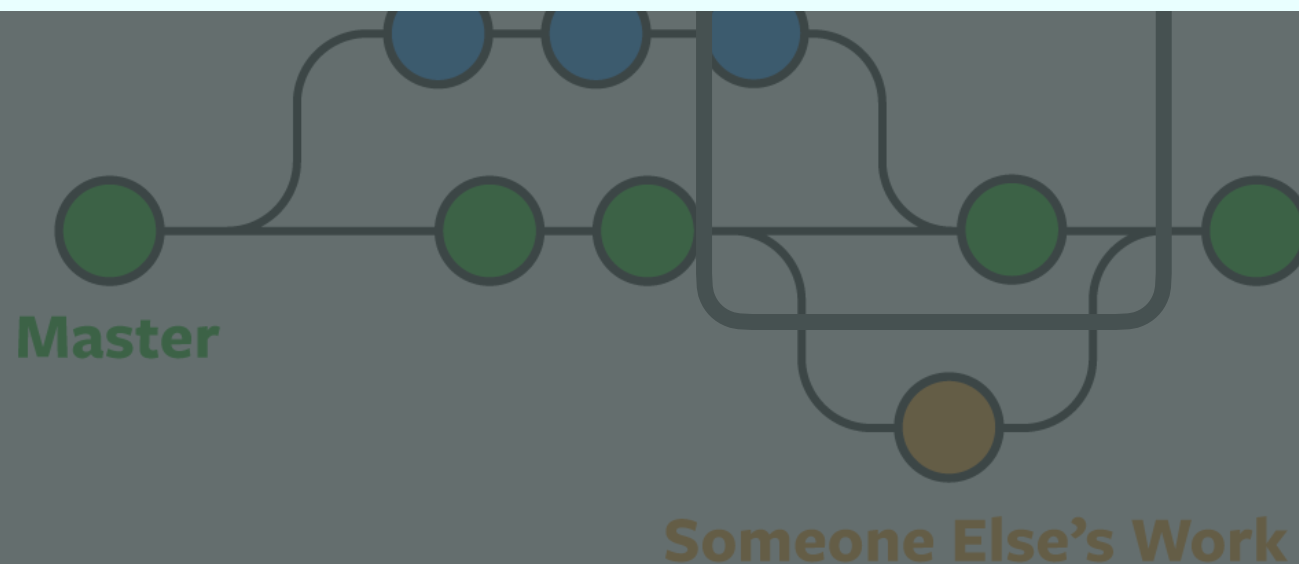
In questo modo si chiederà di “unificare” il branch corrente con quello indicato come dal parametro “merge”

E se volessimo “unire” due branch?

Per “unire” due branch quello che bisogna fare è eseguire il seguente comando:

```
git merge <nome del branch da “unire”>
```

Attenzione che potrebbero avvenire dei conflitti!



In questo modo si chiederà di “unificare” il branch corrente con quello indicato come dal parametro “merge”

Domanda

E se sia io che il mio collega modifichiamo contemporaneamente le stesse linee di codice? Chi di noi due avrebbe la “versione più aggiornata”?

Domanda

E se sia io che il mio collega modifichiamo contemporaneamente le stesse linee di codice? Chi di noi due avrebbe la “versione più aggiornata”?

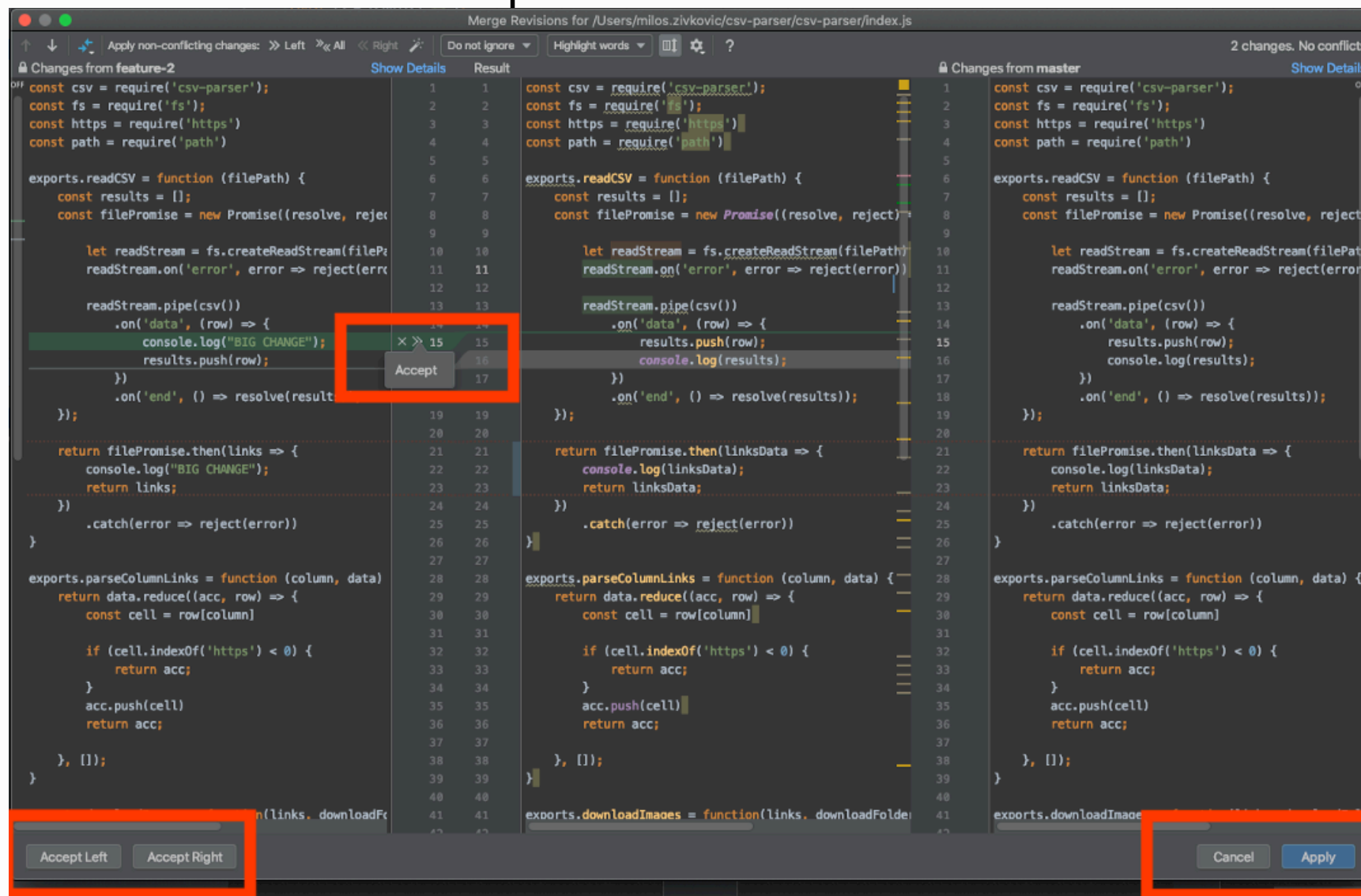
In quel caso git non è in grado di capire qual è la versione più “recente”, quindi chiederà all’ultima persona che ha fatto il comando “push” di “dirglielo”

Domanda

E se sia io che il mio collega modifichiamo contemporaneamente le stesse linee di codice? Chi di noi due avrebbe la “versione più aggiornata”?

In quel caso git non è in grado di capire qual è la versione più “recente”, quindi chiederà all’ultima persona che ha fatto il comando “push” di “dirglielo”

Questa operazione è molto delicata, quindi vi consigliamo di farla utilizzando l’interfaccia grafica che il vostro IDE mette a disposizione

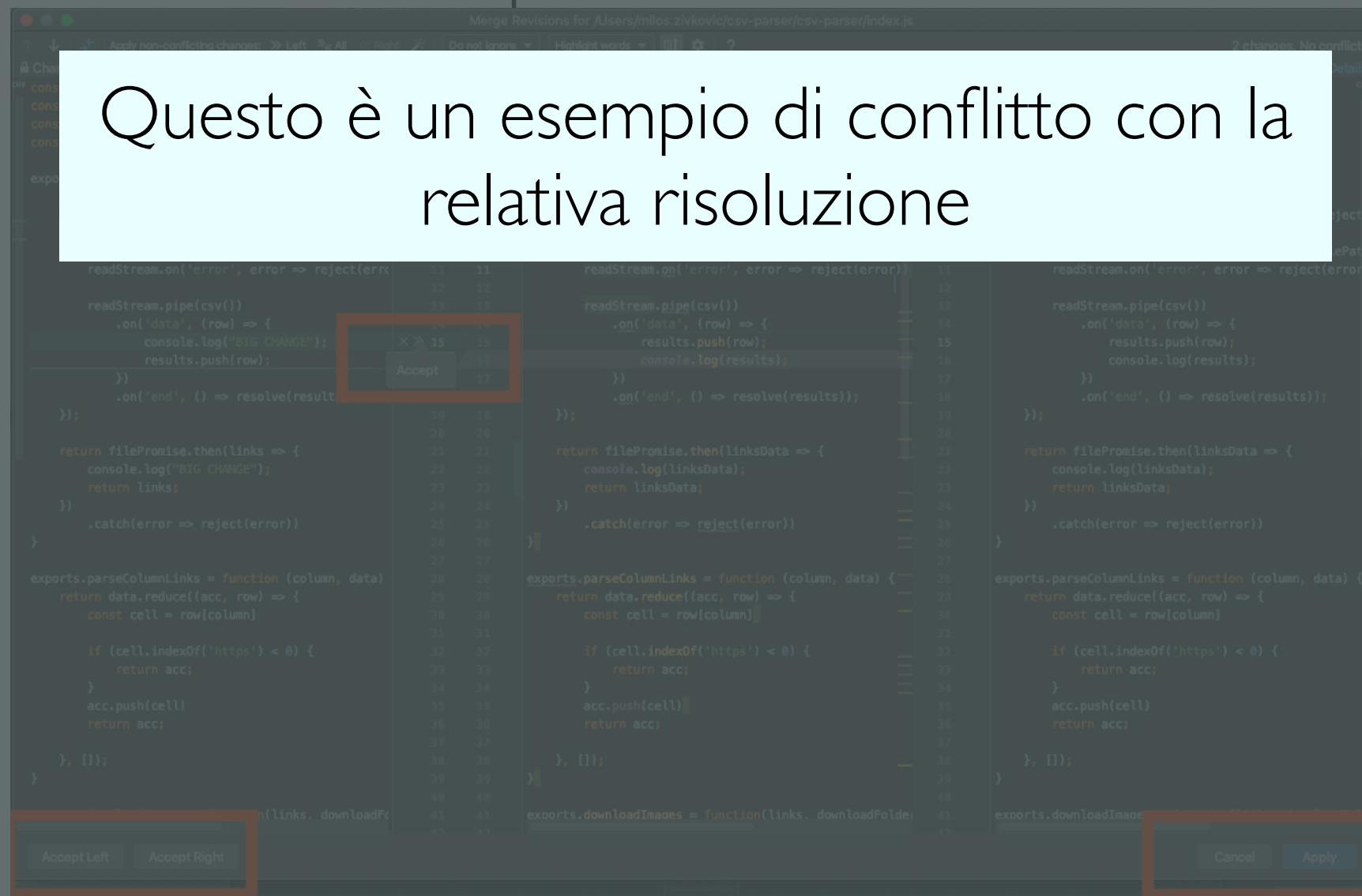


Domanda

E se sia io che il mio collega modifichiamo contemporaneamente le stesse linee di codice? Chi di noi due avrebbe la “versione più aggiornata”?

In quel caso git non è in grado di capire qual è la versione più “recente”, quindi chiederà all’ultima persona che ha fatto il comando “push” di “dirglielo”

Questa operazione è molto delicata, quindi vi consigliamo di farla utilizzando l’interfaccia grafica che il vostro IDE mette a disposizione



Come faccio per essere sicuro di avere l'ultima versione remota disponibile di un progetto?

In questo caso tutto quello che dovrete fare è posizionarvi con il terminale sul progetto e scrivere il seguente comando:

```
git pull
```

Git in automatico verificherà la presenza di nuovi aggiornamenti (e.g., un vostro collega ha apportato qualche modifica a certi file) e in automatico li scaricherà sulla vostra macchina locale

Come faccio per essere sicuro di avere l'ultima versione remota disponibile di un progetto?

In questo caso tutto quello che dovrete fare è posizionarvi con il terminale sul progetto e scrivere il seguente comando:

```
git pull
```

Git in automatico verificherà la presenza di nuovi aggiornamenti (e.g., un vostro collega ha apportato qualche modifica a certi file) e in automatico li scaricherà sulla vostra macchina locale

Consiglio

Per evitare conflitti, ogni volta che decidete di lavorare sul vostro progetto, eseguite SEMPRE una “git pull” prima di iniziare a lavorare, per essere certi di star lavorando sull'ultima versione disponibile.

GITHUB



Cos'è GITHUB?

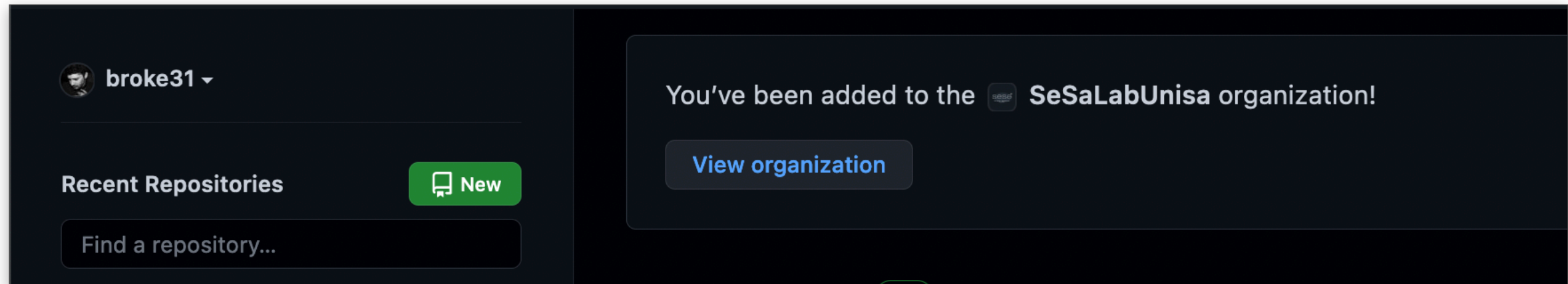
GitHub è un servizio di hosting per progetti software. La caratteristica principale è che tale servizio è una implementazione del sistema di versioning Git

Utilizziamo GITHUB!

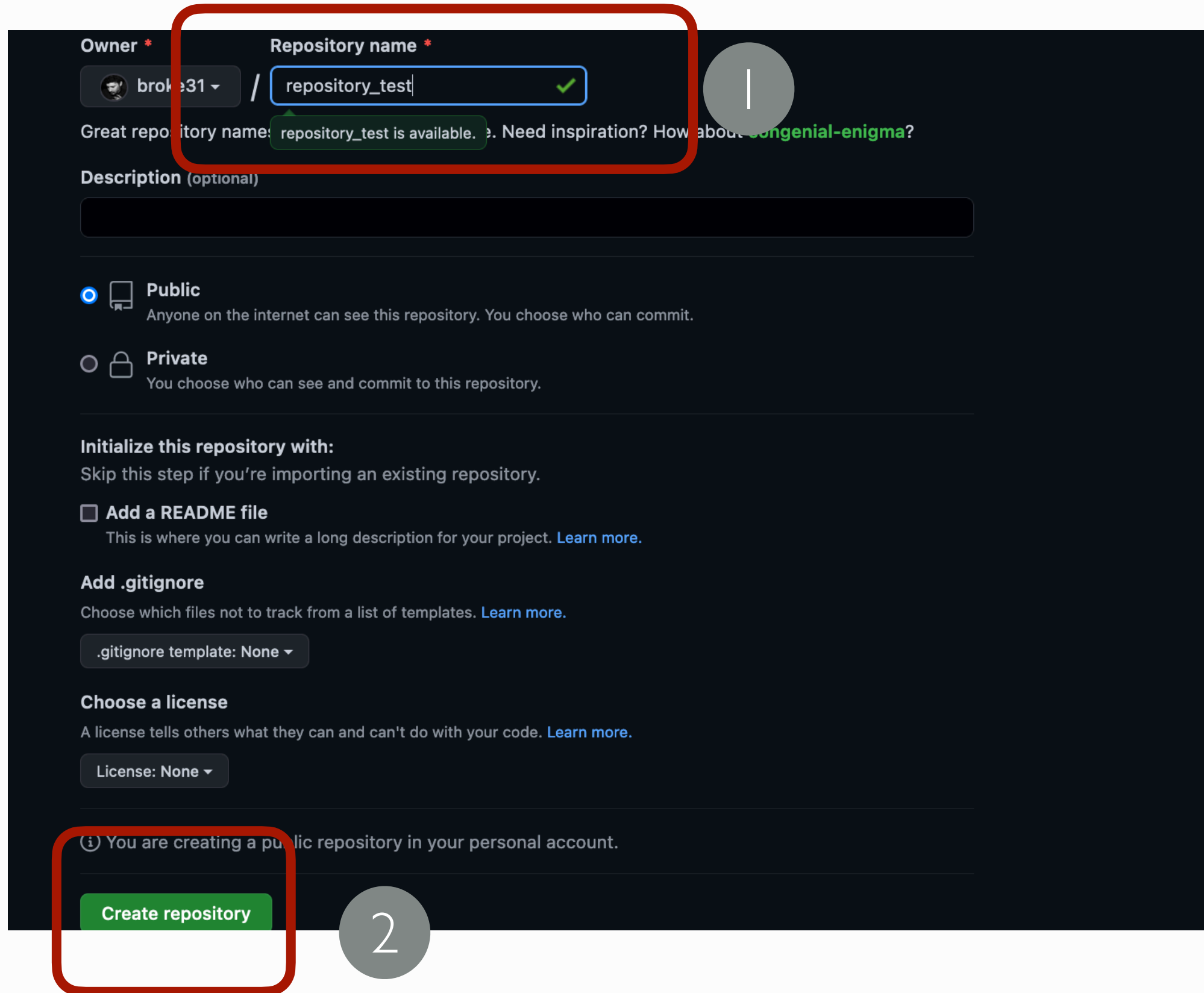
Il primo passo è quello di andare sul sito “www.github.com” e registrarsi

Utilizziamo GITHUB!

Il primo passo è quello di andare sul sito “www.github.com” e registrarsi
Dopo esserci registrati clicchiamo su “new” e seguiamo la procedura guidata per creare una nuova repository



Utilizziamo GITHUB!



The screenshot shows the GitHub 'Create repository' form. A red rounded rectangle highlights the 'Owner' and 'Repository name' fields. The 'Owner' is 'broke31' and the 'Repository name' is 'repository_test', which has a green checkmark. Below the name field, a message says 'Great repository name: repository_test is available.' and suggests 'congenial-enigma?'. A grey circle with the number '1' is next to the repository name field. Below the name field is a 'Description (optional)' text area. The 'Public' option is selected with a radio button, and the 'Private' option is unselected. Below these is the 'Initialize this repository with:' section, which includes a checkbox for 'Add a README file' and a dropdown for 'Add .gitignore'. The 'Choose a license' section has a dropdown for 'License: None'. A red rounded rectangle highlights the 'Create repository' button at the bottom. A grey circle with the number '2' is next to the button. A note at the bottom says '(i) You are creating a public repository in your personal account.'

Owner * / Repository name *

Great repository name: repository_test is available. Need inspiration? How about congenial-enigma?

Description (optional)

☒ Public
Anyone on the internet can see this repository. You choose who can commit.

☐ Private
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☐ Add a README file
This is where you can write a long description for your project. [Learn more.](#)

Add .gitignore
Choose which files not to track from a list of templates. [Learn more.](#)

.gitignore template: None ▾

Choose a license
A license tells others what they can and can't do with your code. [Learn more.](#)

License: None ▾

(i) You are creating a public repository in your personal account.

Create repository

2

Git e Github

Utilizziamo GITHUB!

Quick setup — if you've done this kind of thing before

 Set up in Desktop or **HTTPS** **SSH** `https://github.com/broke31/repository_test.git` 

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# repository_test" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/broke31/repository_test.git
git push -u origin main
```



...or push an existing repository from the command line

```
git remote add origin https://github.com/broke31/repository_test.git
git branch -M main
git push -u origin main
```



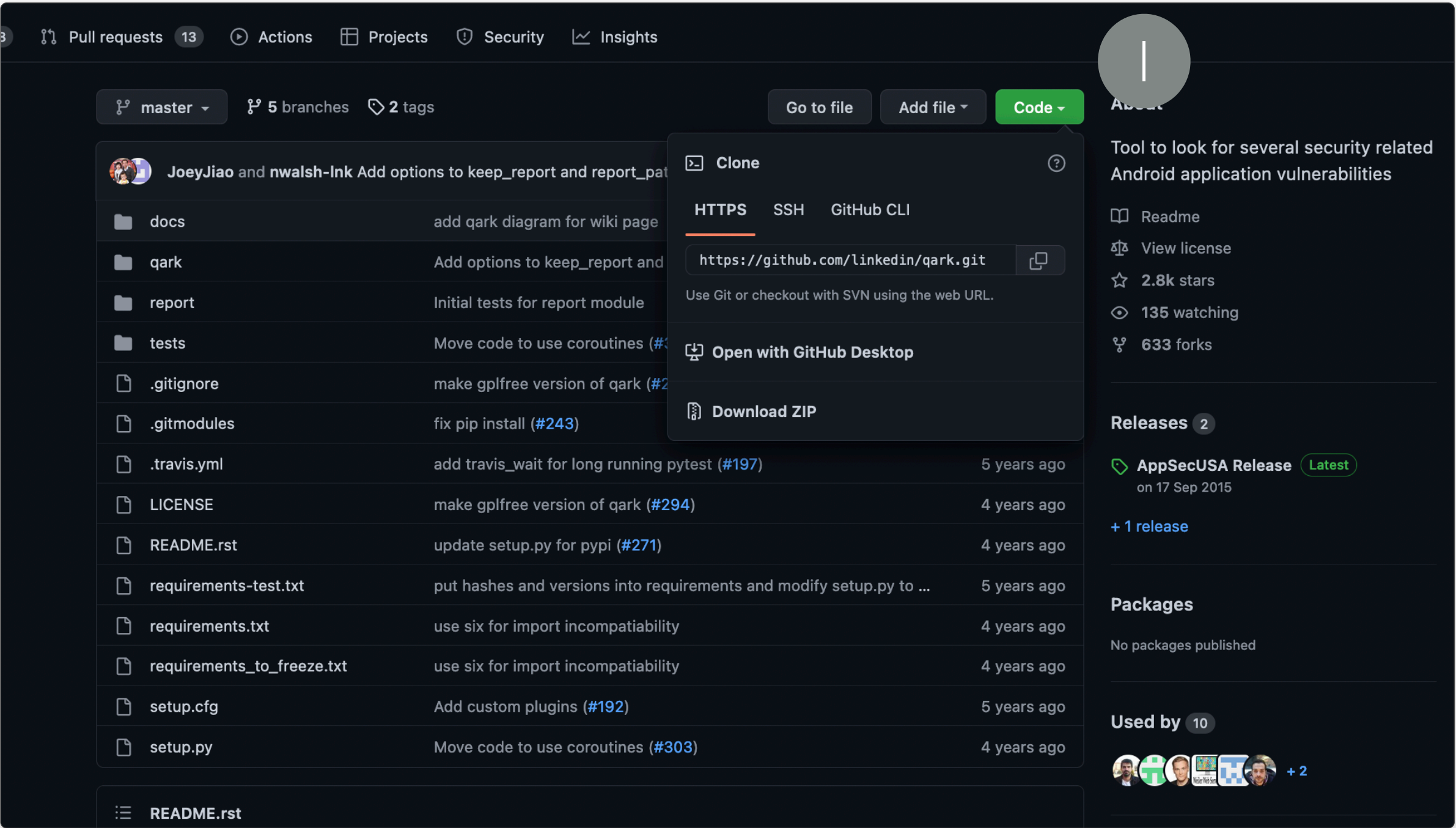
...or import code from another repository

You can initialize this repository with code from a Subversion, Mercurial, or TFS project.

[Import code](#)

Utilizziamo GITHUB!

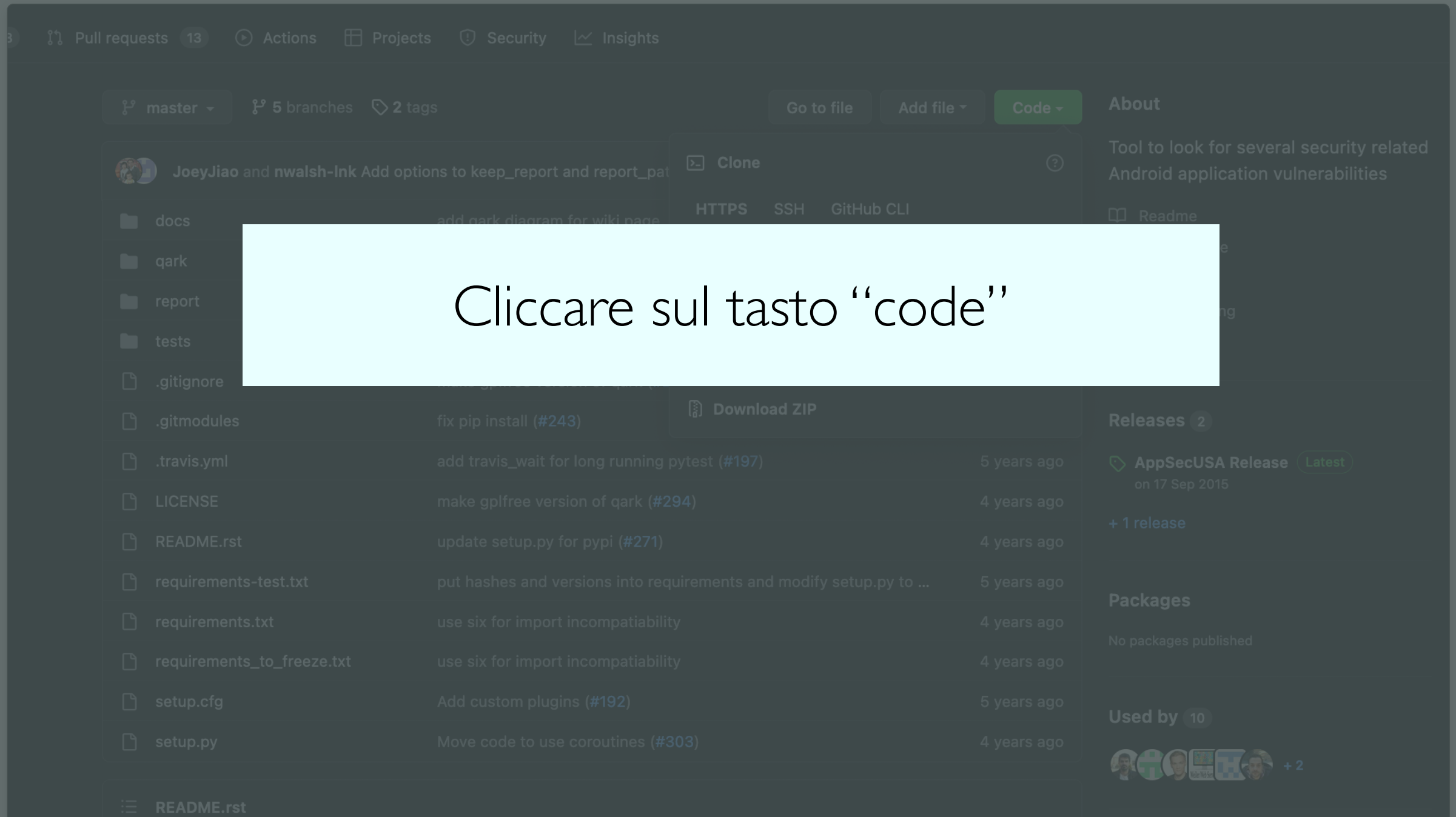
E se volessimo clonare sul nostro pc una repository già esistente?



Utilizziamo GITHUB!

E se volessimo clonare sul nostro pc una repository già esistente?

Cliccare sul tasto “code”



Utilizziamo GITHUB!

E se volessimo clonare sul nostro pc una repository già esistente?

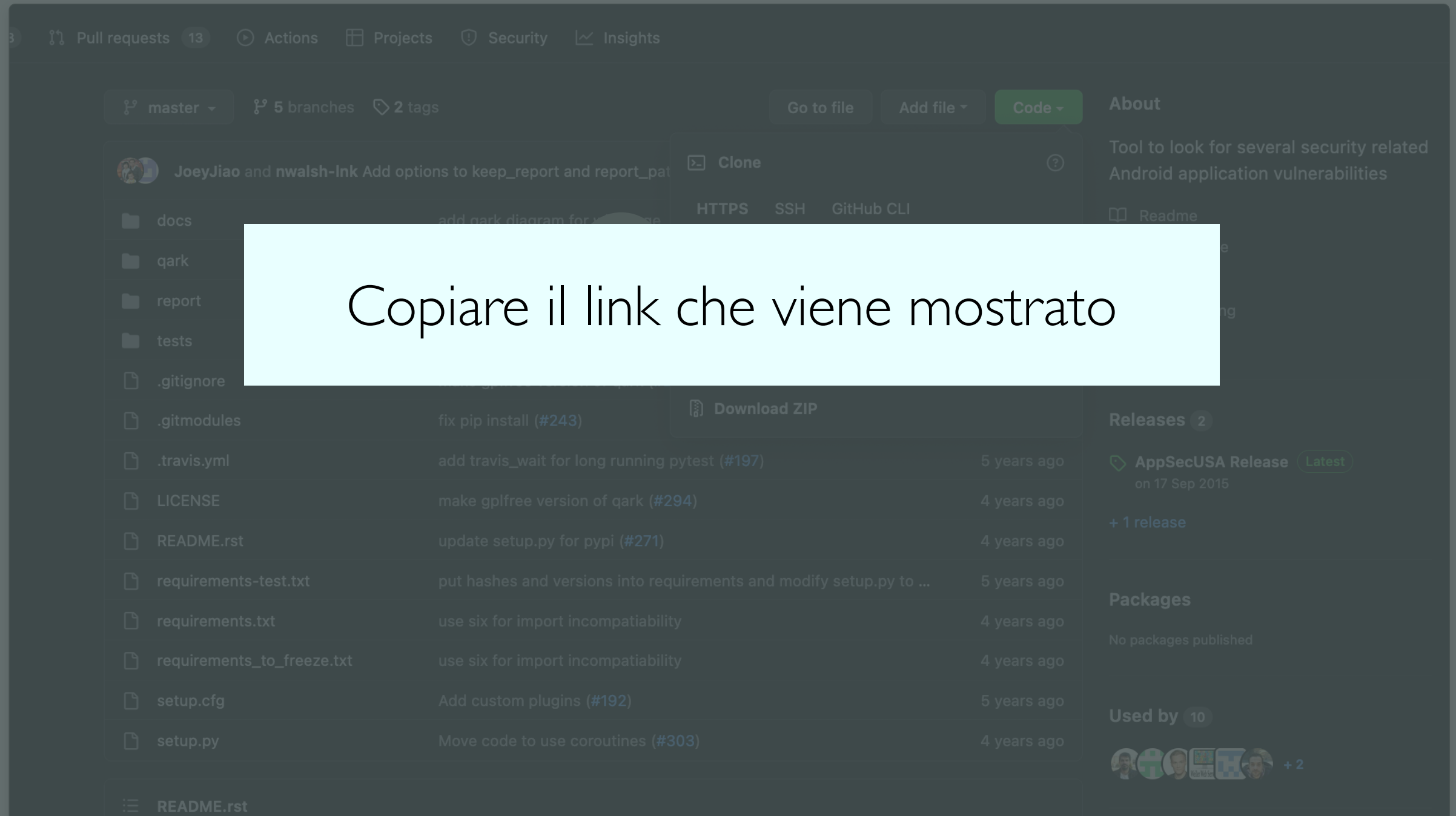
The screenshot shows the GitHub interface for the repository 'qark' by JoeyJiao and nwalsh-Ink. The repository description is 'Tool to look for several security related Android application vulnerabilities'. The repository has 2.8k stars, 135 watching, and 633 forks. The 'Code' button is highlighted with a green circle and the number 1. The 'Clone' dropdown menu is open, showing options for HTTPS, SSH, and GitHub CLI. The HTTPS URL 'https://github.com/linkedin/qark.git' is highlighted with a green circle and the number 2. The repository details on the right show 2.8k stars, 135 watching, and 633 forks.

File	Commit Message	Time Ago
docs	add qark diagram for windows	5 years ago
qark	Add options to keep_report and report_path	5 years ago
report	Initial tests for report module	5 years ago
tests	Move code to use coroutines (#303)	4 years ago
.gitignore	make gplfree version of qark (#294)	4 years ago
.gitmodules	fix pip install (#243)	5 years ago
.travis.yml	add travis_wait for long running pytest (#197)	5 years ago
LICENSE	make gplfree version of qark (#294)	4 years ago
README.rst	update setup.py for pypi (#271)	4 years ago
requirements-test.txt	put hashes and versions into requirements and modify setup.py to ...	5 years ago
requirements.txt	use six for import incompatiability	4 years ago
requirements_to_freeze.txt	use six for import incompatiability	4 years ago
setup.cfg	Add custom plugins (#192)	5 years ago
setup.py	Move code to use coroutines (#303)	4 years ago

Utilizziamo GITHUB!

E se volessimo clonare sul nostro pc una repository già esistente?

Copiare il link che viene mostrato



Utilizziamo GITHUB!

Apriamo un terminale e scriviamo il seguente comando:

```
git clone https://github.com/linkedin/qark.git
```

N.B. Il link da incollare è quello che si è copiato precedentemente!

Utilizziamo GITHUB!

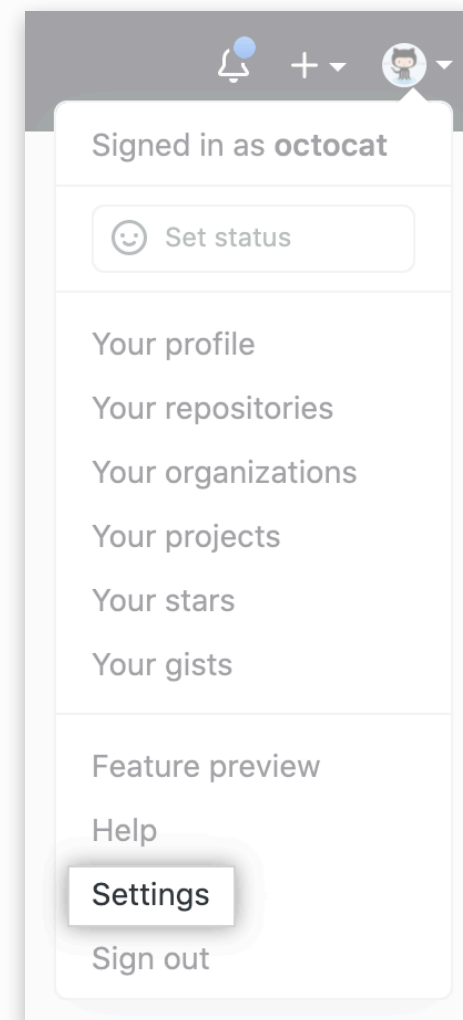
Apriamo un terminale e scriviamo il seguente comando:

```
git clone https://github.com/linkedin/qark.git
```

N.B. Il link da incollare è quello che si è copiato precedentemente!

Se GitHub chiede di inserire una username e una password occorrerà fare la seguente procedura:

Cliccare sul proprio avatar e premere il tasto
“Settings”



Utilizziamo GITHUB!

Apriamo un terminale e scriviamo il seguente comando:

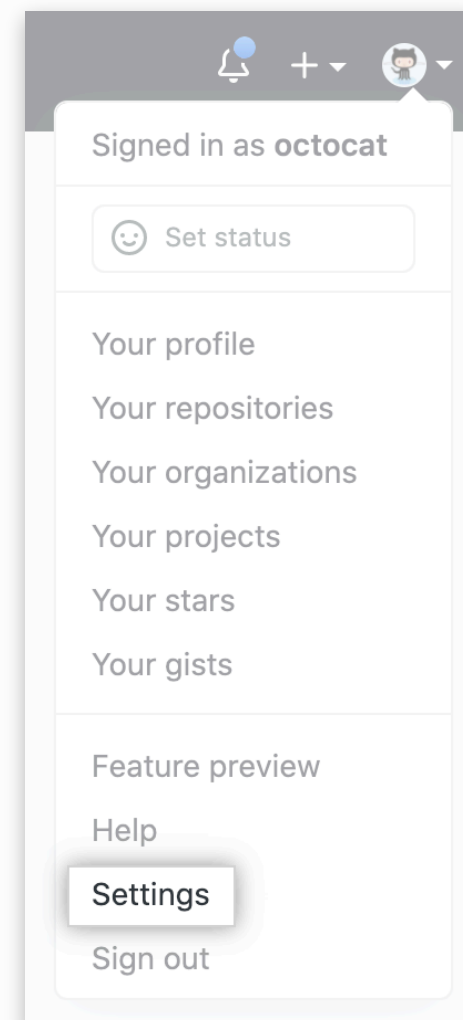
```
git clone https://github.com/linkedin/qark.git
```

N.B. Il link da incollare è quello che si è copiato precedentemente!

Se GitHub chiede di inserire una username e una password occorrerà fare la seguente procedura:

Cliccare sul proprio avatar e premere il tasto “Settings”

Andare sulla sezione “developer settings”



Utilizziamo GITHUB!

Apriamo un terminale e scriviamo il seguente comando:

```
git clone https://github.com/linkedin/qark.git
```

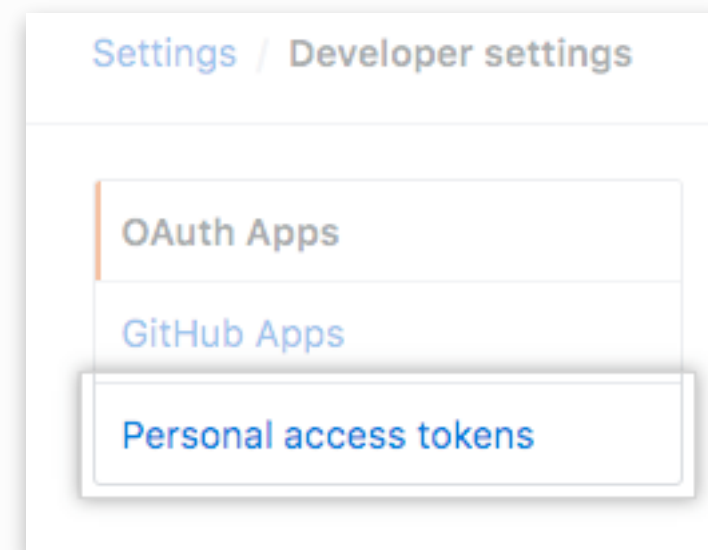
N.B. Il link da incollare è quello che si è copiato precedentemente!

Se GitHub chiede di inserire una username e una password occorrerà fare la seguente procedura:

Cliccare sul proprio avatar e premere il tasto “Settings”

Andare sulla sezione “developer settings”

Cliccare su “Personal access tokens”



Utilizziamo GITHUB!

Apriamo un terminale e scriviamo il seguente comando:

```
git clone https://github.com/linkedin/qark.git
```

N.B. Il link da incollare è quello che si è copiato precedentemente!

Se GitHub chiede di inserire una username e una password occorrerà fare la seguente procedura:

Cliccare sul proprio avatar e premere il tasto
“Settings”

Andare sulla sezione “developer settings”

Cliccare su “Personal access tokens”

Genera nuovo token

Personal access tokens

Generate new token

Utilizziamo GITHUB!

Apriamo un terminale e scriviamo il seguente comando:

```
git clone https://github.com/linkedin/qark.git
```

N.B. Il link da incollare è quello che si è copiato precedentemente!

Se GitHub chiede di inserire una username e una password occorrerà fare la seguente procedura:

Cliccare sul proprio avatar e premere il tasto “Settings”

Andare sulla sezione “developer settings”

Cliccare su “Personal access tokens”

Genera nuovo token

Imposta una data di “scadenza” del token

Expiration

7 days



The token will expire on Friday, Feb 8 2008

Utilizziamo GITHUB!

Apriamo un terminale e scriviamo il seguente comando:

```
git clone https://github.com/linkedin/qark.git
```

N.B. Il link da incollare è quello che si è copiato precedentemente!

Se GitHub chiede di inserire una username e una password occorrerà fare la seguente procedura:

Seleziona i permessi che vuoi dare al token (se hai qualche dubbio puoi dare tutti i permessi che ti vengono chiesti)

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☐ admin:org	Full control of orgs and teams
☐ write:org	Read and write org and team membership
☐ read:org	Read org and team membership
☐ admin:public_key	Full control of user public keys
☐ write:public_key	Write user public keys
☐ read:public_key	Read user public keys
☐ admin:repo_hook	Full control of repository hooks
☐ write:repo_hook	Write repository hooks
☐ read:repo_hook	Read repository hooks
☐ admin:org_hook	Full control of organization hooks
☐ gist	Create gists
☐ notifications	Access notifications
☐ user	Update all user data
☐ user:email	Access user email addresses (read-only)
☐ user:follow	Follow and unfollow users
☒ delete_repo	Delete repositories

Utilizziamo GITHUB!

Apriamo un terminale e scriviamo il seguente comando:

```
git clone https://github.com/linkedin/qark.git
```

N.B. Il link da incollare è quello che si è copiato precedentemente!

Se GitHub chiede di inserire una username e una password occorrerà fare la seguente procedura:

Seleziona i permessi che vuoi dare al token (se hai qualche dubbio puoi dare tutti i permessi che ti vengono chiesti)

Clicca su “genera token”

Utilizziamo GITHUB!

Apriamo un terminale e scriviamo il seguente comando:

```
git clone https://github.com/linkedin/qark.git
```

N.B. Il link da incollare è quello che si è copiato precedentemente!

Se GitHub chiede di inserire una username e una password occorrerà fare la seguente procedura:

Seleziona i permessi che vuoi dare al token (se hai qualche dubbio puoi dare tutti i permessi che ti vengono chiesti)

Clicca su “genera token”

Incolla il token appena generato nel campo “password”

Cos'è una pull request?

Una “pull request” permette di “chiedere il permesso” agli altri sviluppatori quando vogliamo applicare delle modifiche a un progetto di cui non siamo proprietari o collaboratori.

È uno strumento molto utilizzato nell'ambito dell'open source, e consente di comunicare agli altri sviluppatori le modifiche che abbiamo apportato a un ramo di un repository su GitHub. Una volta aperta una richiesta di pull, è possibile discuterne e, dopo che i responsabili del progetto avranno approvato le modifiche, queste saranno integrate nella repository.

Cos'è una pull request?

Una “pull request” permette di “chiedere il permesso” agli altri sviluppatori quando vogliamo applicare delle modifiche a un progetto di cui non siamo proprietari o collaboratori.

È uno strumento molto utilizzato nell'ambito dell'open source, e consente di comunicare agli altri sviluppatori le modifiche che abbiamo apportato a un ramo di un repository su GitHub. Una volta aperta una richiesta di pull, è possibile discuterne e, dopo che i responsabili del progetto avranno approvato le modifiche, queste saranno integrate nella repository.

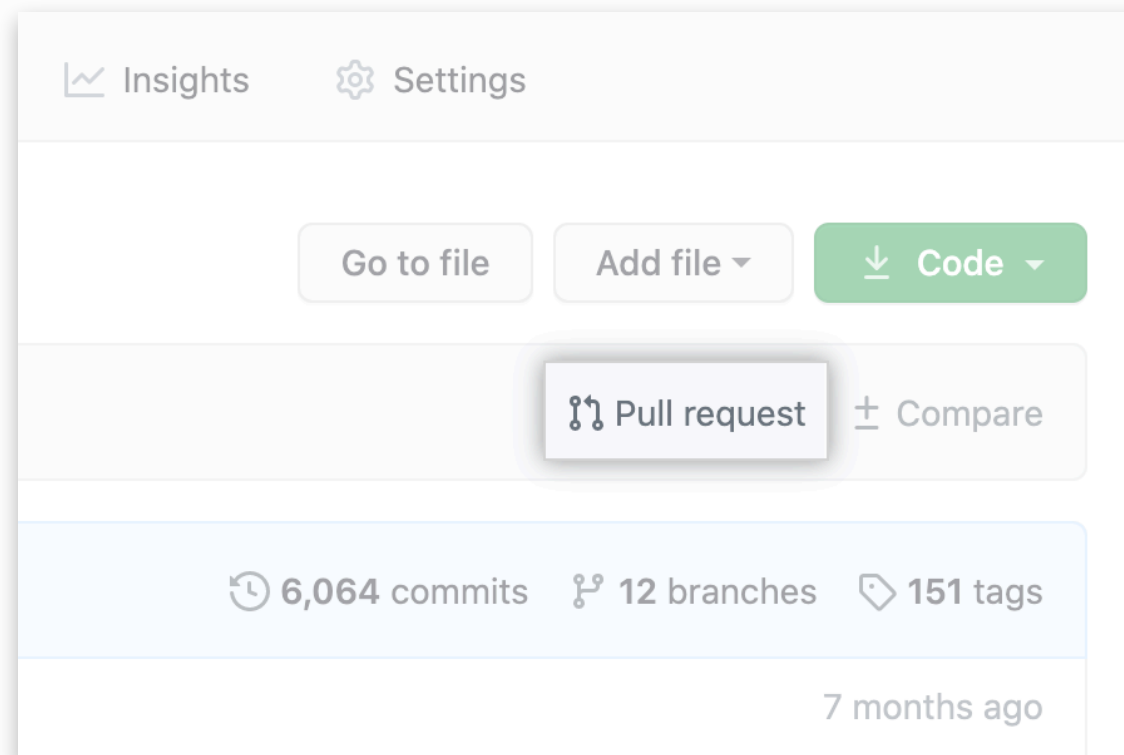
Commit & Push VS Pull Request

Possiamo eseguire Commit & Push su un progetto del quale siamo proprietari (abbiamo dunque tutti i permessi per modificare il codice). Quando vogliamo contribuire al progetto di qualcun altro, dobbiamo utilizzare le Pull Request per “avere il permesso” di aggiungere il nostro codice.

Creiamo una pull request!

Vai su GitHub e seleziona la repository su cui effettuare la tua pull request

Allineati al branch su cui vuoi effettuare la tua pull request (da interfaccia grafica) e premi il pulsante “pull request”



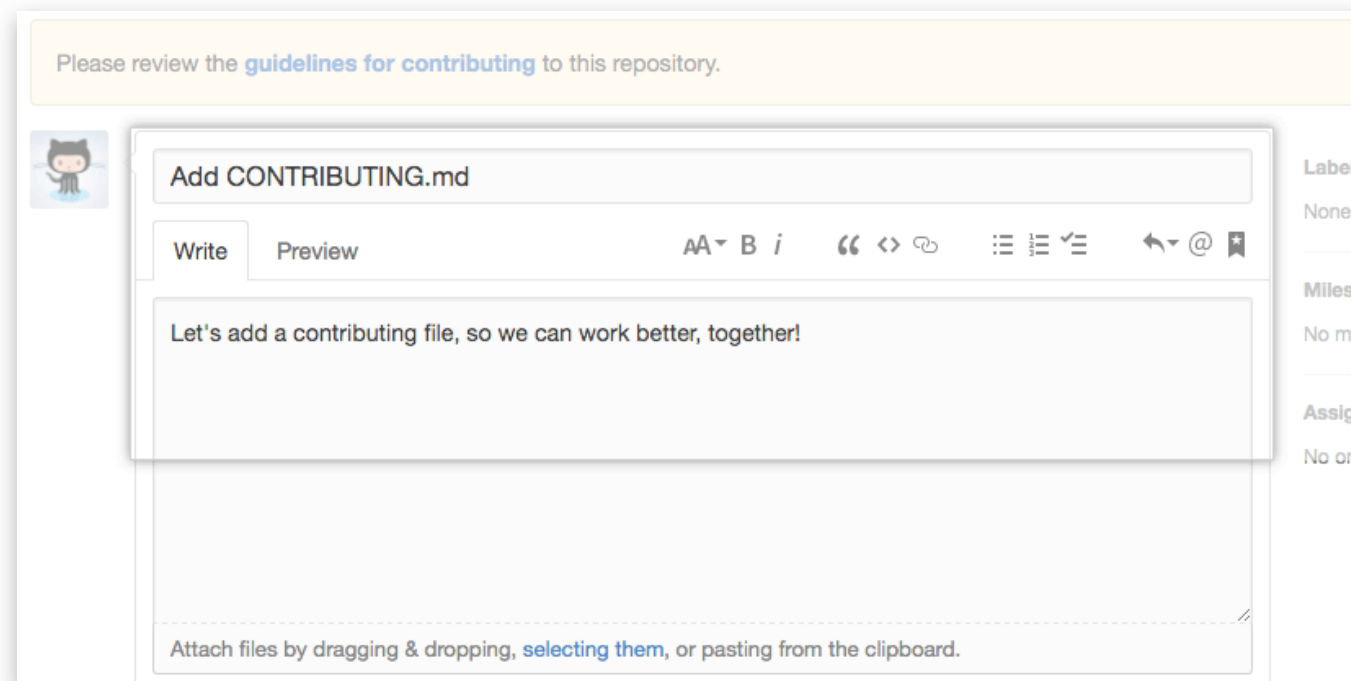
Creiamo una pull request!

Vai su GitHub e seleziona la repository su cui effettuare la tua pull request

Allineati al branch su cui vuoi effettuare la tua pull request (da interfaccia grafica) e premi il pulsante “pull request”

Usa il menu a tendina per selezionare il branch su cui si desidera unire le modifiche, quindi usa il menu a tendina del ramo di confronto per scegliere branch su cui sono state effettuate le modifiche

Inserisci il titolo e la descrizione della pull request che vuoi fare



The screenshot shows a GitHub interface with a yellow banner at the top that reads: "Please review the [guidelines for contributing](#) to this repository." Below this is a modal dialog box titled "Add CONTRIBUTING.md". The dialog has a "Write" tab selected and a "Preview" tab. The "Write" tab contains a text area with the text: "Let's add a contributing file, so we can work better, together!". Above the text area is a rich text editor toolbar with icons for bold, italic, link, unlink, list, and other formatting options. Below the text area is a dashed line indicating where to attach files. At the bottom of the dialog, there is a note: "Attach files by dragging & dropping, [selecting them](#), or pasting from the clipboard." The background of the page shows a sidebar with sections for "Labels", "Milestones", and "Assignees", each with a "None yet" or "No one" status.

Creiamo una pull request!

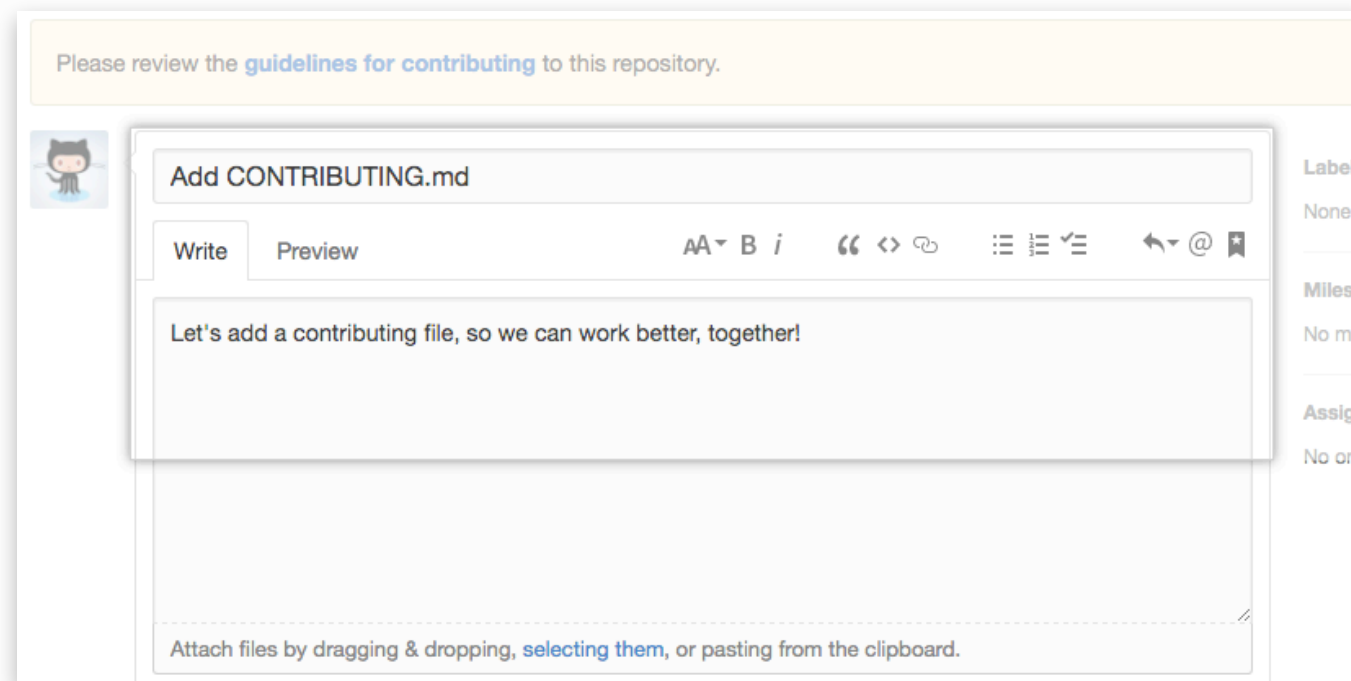
Vai su GitHub e seleziona la repository su cui effettuare la tua pull request

Allineati al branch su cui vuoi effettuare la tua pull request (da interfaccia grafica) e premi il pulsante “pull request”

Usa il menu a tendina per selezionare il branch su cui si desidera unire le modifiche, quindi usa il menu a tendina del ramo di confronto per scegliere branch su cui sono state effettuate le modifiche

Inserisci il titolo e la descrizione della pull request che vuoi fare

Premi sul tasto “Create pull request”



The screenshot shows a GitHub interface for adding a new file named 'CONTRIBUTING.md'. At the top, a yellow banner reads: 'Please review the [guidelines for contributing](#) to this repository.' Below this is a text input field containing the title 'Add CONTRIBUTING.md'. Underneath the title are two tabs: 'Write' (selected) and 'Preview'. To the right of the tabs is a rich text editor toolbar with icons for bold, italic, quote, code, link, list, and other formatting options. The main text area contains the draft description: 'Let's add a contributing file, so we can work better, together!'. At the bottom of the form, there is a dashed line indicating where to attach files, with the text: 'Attach files by dragging & dropping, [selecting them](#), or pasting from the clipboard.' On the right side of the dialog, a sidebar is partially visible with sections for 'Labels', 'Milestones', and 'Assignees', each showing 'None yet' or 'No one assigned'.

In case of fire



1. `git commit`



2. `git push`



3. leave building

Git e GitHub

Laboratorio di Ingegneria del Software

